A DNA-Based Solution to the Subgraph Isomorphism Problem

Sun-Yuan Hsieh* and Chao-Wen Huang †
Department of Computer Science and Information Engineering,
National Cheng Kung University,
No. 1, University Road, Tainan 70101, TAIWAN

Abstract

Since the experimental demonstration of its feasibility, DNA-based computing has been applied to a number of decision or combinatorial optimization problems. The graph isomorphism problem belongs to the class of NP problems, and has been conjectured intractable, although probably not NP-complete. In this paper, we demonstrate the power of DNA-based computing by showing the subgraph isomorphism problem can be efficiently solved under this computation model. By generating the solution space using stickers, we present DNA-based algorithms to solve the problem using polynomial number of basic biological operations.

1 Introduction

DNA-based computing is an exciting research area at the intersection of mathematics, computer science and molecular biology. Feynman [11] first proposed DNA-based computation in 1961, but his idea was not implemented by experiment for a few decades. By properly manipulating DNA strands as the input instance of the Hamiltonian path problem, Adleman [1] succeeded in solving the problem in a test tube. Since then, many researchers studied on computational hard problems using Adleman-type methods [2, 3, 4, 5, 10, 12, 13, 15, 16, 17, 19]. In particular, Lipton [15] demonstrated the power of DNA-based computing using the Adleman techniques to solve the satisfiability problem (the first NP-complete problem). Roweis *et al.* [18] proposed the concept of *sticker* for enhancing the Adleman-Lipton model. Several NP-complete problems were solved using the Adleman-Lipton model with stickers (for example, see [6, 7, 8, 9, 14, 18]). However, there are quite a few NP-complete problems that do not seem to be solvable using this model because it is very difficult to support basic biological operations using complex mathematical operations. That is, we are not sure whether DNA-based computing can be applied to dealing with every NP-complete problem.

A *graph* $G = (V, E)$ is a pair of the *vertex set* V and the *edge set* E, where V is a finite set and E is a subset of $\{(u, v) \mid (u, v)$ is an unordered pair of $V\}$. We also use $V(G)$ and $E(G)$ to denote the vertex set and the edge set of G, respectively. If (u, v) is an edge in G, we say that u and v are *adjacent*. An *isomorphism* from a simple graph G to a simple graph H is a one-to-one and onto function $\pi : V(G) \rightarrow V(H)$ such that $(u, v) \in E(G)$ if and only if $(\pi(u), \pi(v)) \in E(H)$. We say "$G$ is *isomorphic to* H", written $G \cong H$, if there is an isomorphism from G to H. Given two graphs $G = (V, E)$ and $G_s = (V', E')$, iff $V' \subseteq V$ and $E' \subseteq E \wedge ((v_1, v_2) \in E' \rightarrow v_1, v_2 \in V')$, we say that the graph G_s is the subgraph of G, denoted by $G_s \subseteq G$. The *subgraph isomorphism problem* is to determine whether G_s is isomorphic to H. In this paper, we call H as the *target graph*. The subgraph isomorphism problem is an important problem that is known to be NP-complete.

In this paper, we first use stickers to construct the solution space for the subgraph isomorphism problem. By utilizing biological operations in the Adleman-Lipton model, we develop DNA-based algorithms for parallel generating $n!$ permutations of $\{1, 2, \ldots, n\}$ and representing adjacency relation of a given graph. We also present a DNA-based algorithm to achieve parallel comparative operation. Based on these algorithms, we prove that the subgraph isomorphism problem can be efficiently solved using the biological operations in the Adleman-Lipton model with stickers. Our result provides clear evidence of the ability of DNA-based computing to solve the problem which is currently unknown to be in P or in NP-complete.

*To whom correspondence should be addressed. E-mail: hsiehsy@mail.ncku.edu.tw (Professor Sun-Yuan Hsieh)
†E-mail: huangcw@algorithm.csie.ncku.edu.tw

This paper is organized as follows. In Section 2, we introduces the Adleman-Lipton model in details. In Section 3, we present DNA-based algorithms to generate the solution space for the subgraph isomorphism problem. In Section 4, a complete algorithm for the problem is presented. Discussion and conclusion are given in Section 5.

2 The Adleman-Liption model

A *set* is a group of objects represented as a unit. If we do want to take the number of occurrences of members into account, we call the group a *multi-set* instead of a set. In Adleman-Liption model [1], a tube is a multi-set of DNA strands (single-strands) over an alphabet set $\{\mathbf{A}, \mathbf{G}, \mathbf{C}, \mathbf{T}\}$.[1] Given a tube, one can perform the following operations:

1. Extract(T, S): Given a tube T and a short single strand of DNA, say S, the operation produces two tubes $(T, S)^+$ and $(T, S)^-$, where $(T, S)^+$ consists of all the molecules of DNA in T such that each contains S as a sub-strand, and $(T, S)^-$ consists of all the molecules of DNA in T which do not contain S. Finally, the tube T becomes empty.

2. Merge(T_1, T_2): Given tubes T_1 and T_2, the operation is to pour two tubes into one, without any change in the individual strands. We also use $T_1 \cup T_2$ to denote the resulting molecules after the operation.

3. Detect(T): Given a tube T, the operation returns "yes" if tube T contains at least one DNA molecule. Otherwise, it returns "no".

4. Amplify(T, T_1, T_2): Given a tube T, the operation produces two tubes T_1 and T_2 such that T_1 and T_2 contain the original copy of those molecules in T and then tube T becomes empty after this operation.

5. Read(T): Given a tube T, the operation is to describe a single molecular contained in T. Moreover, the operation can give an explicit description of exactly one of them even if T contains many different molecules each encoding a different set of bases.

6. Append(T, S): Given a tube T and a short DNA strand S, the operation appends S onto the end of every strand in T.

7. Discard(T): Given a tube T, the operation discards all the molecules in T.

[1] Distinct nucleotides are detected only with their bases, which come in two sorts: *purines* and *pyrimidines*. Purines include *adenine* and *guanine*, abbreviated **A** and **G**. Pyrimidines contain *cytosine* and *thymine*, abbreviated **C** and **T**. Because nucleotides are distinguished solely from their bases, they are simply represented as **A**, **G**, **C**, or **T** nucleotides, depending upon the kinds of base that they have.

3 Sticker-based solution space

Given a positive integer i, let $binary(i)$ denote the "reverse" binary representation $b_1 b_2 \dots b_k$ of i, where $k = \lceil \log_2(i+1) \rceil$. We refer to the leftmost bit b_1 as the *least significant bit* (*LSB* for short) and the rightmost bit b_k as the *most significant bit* (*MSB* for short). For example, $binary(8) = 0001$. In the remainder of this paper, we assume that the vertices of an n-vertex graph are represented by $\{1, 2, \dots, n\}$ such that each vertex has a unique label in $\{1, 2, \dots, n\}$.

Given the set $\{1, 2, \dots, n\}$, any linear arrangement of these n numbers is called an *n-permutation*. The set of all the n-permutations is called the *n-permutation set*, denoted by $\mathcal{P}(n)$.

3.1 Generating the permutation set associated with the adjacency relation

We use notation x^0 (respectively, x^1) to denote a 15-base sticker representing 0 (respectively, 1). In our algorithms, the symbol (integer) i in a permutation is represented in its reverse binary representation using stickers. For example, 123 in a 3-permutation is represented by $x^1x^0 \parallel x^0x^1 \parallel x^1x^1 (= x^1x^0x^0x^1x^1x^1)$, where "$\parallel$" means the string-concatenation operation. We say that $x^1x^0x^0x^1x^1x^1$ is a *DNA-representation* of 123. In the remainder of this paper, for any permutation we refers to is its DNA-representation. We utilize stickers to represent the adjacency relation of the given n-vertex graph G for every n-permutation. We attempt to append $\frac{n(n-1)}{2}$ 15-base stickers $y_1 y_2 \dots y_{\frac{n(n-1)}{2}}$ to every n-permutation $\alpha_1 \alpha_2 \dots \alpha_n$, where $\alpha_1, \alpha_2, \dots, \alpha_n$ are n vertices of G, such that the following two conditions hold:

C1: $\frac{n(n-1)}{2}$ 15-base stickers $y_1, y_2, \dots, y_{\frac{n(n-1)}{2}}$ represent the adjacency relation of $\frac{n(n-1)}{2}$ pairs of vertices $(\alpha_2, \alpha_1), (\alpha_3, \alpha_1), (\alpha_3, \alpha_2), (\alpha_4, \alpha_1), (\alpha_4, \alpha_2)$,

$(\alpha_4, \alpha_3), \ldots, (\alpha_n, \alpha_1), (\alpha_n, \alpha_2), \ldots, (\alpha_n, \alpha_{n-1})$, respectively. In general, y_i represents the adjacency relation of (α_d, α_c), where $i = \frac{(d-1)(d-2)}{2} + c$.

C2:

$$y_{\frac{(d-1)(d-2)}{2}+c} = \begin{cases} 1 & \text{if } \alpha_c \text{ and } \alpha_d \text{ are adjacent} \\ 0 & \text{otherwise} \end{cases}$$

Lemma 1. [20] *Given an n-vertex graph G and a tube T_{adj}, all n-permutations associated with the adjacency relation of G satisfying Conditions C1 and C2 can be generated in T_{adj}.*

3.2 Comparing two graphs

Given an n-vertex graph G, the n-permutation set $\mathcal{P}(n)$ of G can be easily generated by lemma 1. Let $P = \alpha_1\alpha_2 \ldots \alpha_n$ be one n-permutation of $\mathcal{P}(n)$ and $Y = y_1y_2 \ldots y_{\frac{n(n-1)}{2}}$ is adjacency relation of P. We use PY to denote the *string-representation* of the graph G. Clearly, the graph G also can be drawn by the string-representation because the vertex set and edge set of G can obtain by PY. We then use the following algorithms to contrast adjacency relations of two graphs.

Lemma 2. *Given two n-vertex graphs G and H, P=$\alpha_1\alpha_2 \ldots \alpha_n$ and $P' = \alpha'_1\alpha'_2 \ldots \alpha'_n$ are n-permutations of G and H, respectively, and $Y = y_1y_2 \ldots y_{\frac{n(n-1)}{2}}$ and $Y' = y'_1y'_2 \ldots y'_{\frac{n(n-1)}{2}}$ represent the adjacency relations of P and P′, respectively. If $Y = Y'$, we have $\pi : V(G) \to V(H)$ such that $\pi(\alpha_1) = \alpha'_1$, $\pi(\alpha_2) = \alpha'_2, \ldots, \pi(\alpha_n) = \alpha'_n$, and the graph G is isomorphic to the graph H.*

Proof. Let $\pi(\alpha_i) = \alpha'_i$ for $i = 1, 2, \ldots, n$, and y_i and y'_i represent the adjacency relations of (α_d, α_c) and (α'_d, α'_c). Because of $Y = Y'$, we have $y_i = y'_i$ for $i = 1, 2, \ldots, \frac{n(n-1)}{2}$. By definition, the graph G is isomorphic to the graph H. □

Lemma 3. *Given two n-vertex graphs G and H, $P = \alpha_1\alpha_2 \ldots \alpha_n$ and $P' = \alpha'_1\alpha'_2 \ldots \alpha'_n$ are n-permutations of G and H, respectively, and $Y = y_1y_2 \ldots y_{\frac{n(n-1)}{2}}$ and $Y' = y'_1y'_2 \ldots y'_{\frac{n(n-1)}{2}}$ represent the adjacency relations of P and P′, respectively. If each $y_i^1{}'$ in H has the corresponding y_i^1 in G, there exists a subgraph G_s in G, such that G_s is isomorphic to H, and $\pi(\alpha_1) = \alpha'_1, \pi(\alpha_2) = \alpha'_2, \ldots, \pi(\alpha_n) = \alpha'_n$.*

Proof. First, we construct the string-representation of the subgraph G_s by $\alpha_1\alpha_2 \ldots \alpha_n \parallel [(y_1y_2 \ldots y_{\frac{n(n-1)}{2}})\&\&(y'_1y'_2 \ldots y'_{\frac{n(n-1)}{2}})]$, where "$\parallel$" means string-concatenation operation and "&&" means binary AND operation. Clearly, the binary AND operation only changes y_j from 1 to 0 in G, when y_j is 1 in G but its corresponding y'_j is 0 in H, $j = 1, 2, \ldots, \frac{n(n-1)}{2}$. Therefore, the string-representation of the G_s is $\alpha_1\alpha_2 \ldots \alpha_n y'_1y'_2 \ldots y'_{\frac{n(n-1)}{2}}$, and G_s is also the subgraph of G. By lemma 2 , G_s is isomorphic to H, and $\pi(\alpha_1) = \alpha'_1, \ldots, \pi(\alpha_n) = \alpha'_n$. □

Because of the same number of vertices, the number of the adjacency representation is also the same, we say that the two graphs G and H can be *aligned*. By lemma 2 and lemma 3, we can easily determine whether two n-vertex graphs G(or its n-vertex subgraph G_s) and H are isomorphic. In brief, if the string-representation of the graph G is PY , where P is the n-permutation of G and Y is the adjacency representation of P. The same, the string-representation of graph H is $P'Y'$. The graphs G(or G_s) and H can be determined whether they are isomorphic by *aligning* Y and Y'. The achievement of *alignment* is to compare the bits of Y and Y' one by one.

Example 1. In Figure 1, the string-representation of G_1 is PY = 123451111011101, and the string-representation of G_2 is $P'Y'$ = 1′2′3′4′5′1010011101. By aligning Y and Y', each $y_i^1{}'$ in G_2 has the corresponding y_i^1 in G_1, we can construct a graph G_3 by $P\|(Y\&\&Y')$ = 123451010011101. Clearly, the graph G_3 is formed with removing the edges (1,3) and (1,4) from G_1, and the graph G_3 is still a subgraph of the graph G_1. By lemma 2, the adjacency representations of G_2 and G_3 are the same, we have $G_2 \cong G_3$. There exists an isomorphism $\pi : V(G_2) \to V(G_3)$ such that $\pi(1') = 1, \pi(2') = 2, \pi(3') = 3, \pi(4') = 4$, and $\pi(5') = 5$.

Given n_1-vertex graph G and n_2-vertex graph H, without loss of generality, let $n_1 > n_2$. Now, we show how to determine whether $G_s \subseteq G$ is isomorphic to H. Because of $n_1 > n_2$, the string-representations of G and H can't be aligned. In this case, we introduce the conception of *vertex-extension*. The vertex-extension is to add $(n_1 - n_2)$ *pseudo vertices* $\{X_1, X_2, \ldots, X_{n_1-n_2}\}$ into the graph H. Here, the pseudo vertices don't really exist, they only assist us in aligning the two repre-

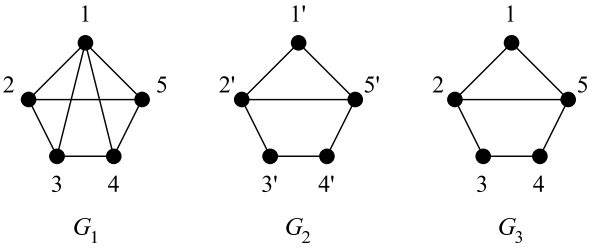

Figure 1: G_3 is the subgraph of G_1, and G_3 is isomorphic to G_2.

sentation of G and H. Moreover, there are no edges connected pseudo vertices and original vertices in H. The resulting graph is called vertex-extension graph of H, labeled H_e.

Algorithm 1 Vertex_Extension(H, n_1, n_2)

Input: (1) H is an n_2-vertex target graph. (2) n_1 is the number of vertices of G. (3) n_2 is the number of vertices of H.

Output: H_e is an n_1-vertex graph .

1: **if** $n_1 > n_2$ **then**
2: Let H_e be the graph H adding (n_1-n_2) pseudo vertices, labeled $X_1, X_2, \ldots, X_{n_1-n_2}$.
3: **else**
4: H_e=H
5: **end if**
6: Return H_e

Example 2. In Figure 2, the graphs G and H are 5-vertex and 4-vertex graph, respectively. Given two permutations $P = 35124$ and $P' = 1'2'3'4'$ are vertex permutations of G and H, respectively. The adjacency representations of P and P' are $Y = 0011111100$ and $Y' = 000111$, respectively. Apparently, the string-representations $PY = 351240011111100$ and $P'Y' = 1'2'3'4'000111$ can't be aligned because of the different length. In order to align PY and $P'Y'$, we add one pseudo vertex, labeled X_1, into the graph H, the resulting graph shows in H_e. The permutation P'' of H_e is $1'2'3'4'X_1$. Undoubtedly, the graph H is isomorphic to H_e except the pseudo vertex X_1. Therefore, if any graph is isomorphic to the graph H_e, it is also isomorphic to H except the corresponding vertex of X_1. Because there are no edges between X_1 and the original vertices in H, the adjacency relations between X_1 and the original vertices in H are represented by all 0's, that is, the adjacency representation of P'' is $Y'' = 0001110000$. Now, we have $PY = 351240011111100$ and $P''Y'' = 1'2'3'4'X_10001110000$. By aligning PY of G and $P''Y''$ of H_e, we have each y_i^1" in H_e has the corresponding y_i^1 in G. By lemma 3, there exists $G_s \subseteq G$ such that G_s is isomorphic to H_e. We construct the subgraph G_s by $PY'' = 351240001110000$, and the resulting graph G_s is shown on the most right side in Figure 2. Moreover, because the vertex X_1 is a pseudo vertex, we let its corresponding vertex '4' in G_s change into pseudo vertex. Then, we have an isomorphism $\pi_1 : V(H_e) \to V(G_s)$ such that $\pi_1(1') = 3, \pi_1(2') = 5, \pi_1(3') = 1, \pi_1(4') = 2$, and $\pi_1(X_1) = 4$. Clearly, we also have an isomorphism $\pi_2 : V(H) \to V(G_s)$ such that $\pi_2(1') = 3, \pi_2(2') = 5, \pi_2(3') = 1$, and $\pi_2(4') = 2$. Finally, we have the subgraph $G_s \subseteq G$ is isomorphic to the graph H.

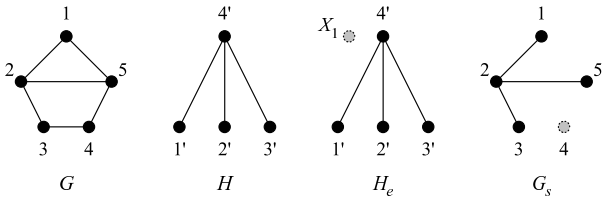

Figure 2: G and H are 5-vertex and 4-vertex graphs, respectively. After executing Vertex_Extension, the resulting graph shows in H_e. The graph G_s is constructed by $PY'' = 351240001110000$, and H_e is isomorphic to G_s.

Given n_1-vertex graphs G and n_2-vertex graphs H, without loss of generality, let $n_1 \geq n_2$. The resulting tube T_{adj} contains $n_1!$ n_1-permutations associated with the adjacency relation of G. We then use the following algorithm to compare the adjacency relation of G with that of H.

Example 3. Consider two graphs G and H shown in Figure 3. Note that $V(H) = \{a, b, c\}$ and $E(H) = \{(a, b), (b, c), (a, c)\}$. Because of $|V(G)| > |V(H)|$, we execute the algorithm Vertex_Extension$(H, 4, 3)$. After executing, the resulting graph is named H_e, $V(H_e) = \{a, b, c, X_1\}$ and $E(H_e) = \{(a, b), (b, c), (a, c)\}$. We can rename the vertices of H with $label(a) = 1'$, $label(b) = 2'$, $label(c) = 3'$ and $label(X_1) = 4'$, and thus the edge set of H is $E(H_e) = \{(1', 2'), (2', 3'), (1', 3')\}$. The graph H_e is shown in Figure 3. Moreover, the target graph H_e associated with its adjacency relation can be represented as $1'2'3'4'y_1'y_2'y_3'y_4'y_5'y_6'$, where $1'2'3'4'$ is a linear arrangement of $V(H_e)$ and $y_1'y_2'y_3'y_4'y_5'y_6'(= 111000)$ is its adjacency relation of H_e. Algorithm

Algorithm 2 Adjacency_Relation_Comparison(H_e,T_{adj}, n_1)

Input: (1) H_e is a target graph. (2) T_{adj} is a tube containing all the n_1-permutations associated with the adjacency relation of G. (3) n_1 is the number of vertices in G.

Output: $T_{result} = \emptyset$ if $G \ncong H_e$, and $T_{result} \neq \emptyset$ if $G \cong H_e$.

1: $T_{result} := T_{adj}$
2: **for** $i = 2$ to n_1 **do**
3: **for** $j = 1$ to $i - 1$ **do**
4: **if** i is adjacency to j in H_e **then**
5: Extract($T_{result}, y^1_{\frac{(i-1)(i-2)}{2}+j}$)
6: $T_a := (T_{result}, y^1_{\frac{(i-1)(i-2)}{2}+j})^+$ and $T_b := (T_{result}, y^1_{\frac{(i-1)(i-2)}{2}+j})^-$
7: $T_{result} := \bigcup(T_{result}, T_a)$
8: Discard(T_b)
9: **end if**
10: **end for**
11: **end for**

Adjacency_Relation_Comparison(H_e,T_{adj},4) compares all the 4-permutations associated with the adjacency relation of G (collected in T_{adj}) with $1'2'3'4'111000$. T_{adj} is shown in the top of Table 1.

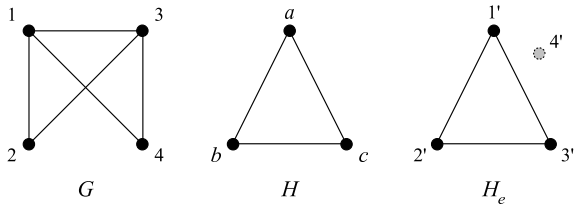

Figure 3: G and H are 4-vertex and 3-vertex graphs, respectively. H_e is vertex-extend graph of H.

For the first iteration on $i = 2$ and $j = 1$, the bit y'_1 of the graph H_e is 1, the **Extract** operation produces two tubes T_a and T_b, where T_a consists of all the molecules of DNA in T_{result} such that each contains y^1_1 as a sub-strand, and T_b consists of all the molecules of DNA in T_{result} which do not contain y^1_1. Next step, the tubes T_{result} and T_a are **Merged** into new tube, also named T_{result}, and T_b is **Discard**. After executing the iteration on $i = 2$ and $j = 1$, DNA strands of the new tube T_{result} is shown in the second table of Table 1. In Table 1, others show the results after executing the remaining iterations.

$T_{result} = T_{adj} =$ (Initial state)	4312111011	3412111101	3142111110	3124111110
	4321101111	3421110111	3241110111	3214111101
	4132111011	1432111101	1342111110	1324111110
	4231011111	2431011111	2341101111	2314111011
	4123101111	1423110111	1243110111	1234111101
	4213011111	2413011111	2143101111	2134111011
$T_{result} =$ (i=2 and j=1)	4312111011	3412111101	3142111110	3124111110
	4321101111	3421110111	3241110111	3214111101
	4132111011	1432111101	1342111110	1324111110
			2341101111	2314111011
	4123101111	1423110111	1243110111	1234111101
			2143101111	2134111011
$T_{result} =$ (i=3 and j=1)	4312111011	3412111101	3142111110	3124111110
		3421110111	3241110111	3214111101
	4132111011	1432111101	1342111110	1324111110
				2314111011
		1423110111	1243110111	1234111101
				2134111011
$T_{result} =$ (i=3 and j=2)	4312111011	3412111101	3142111110	3124111110
				3214111101
	4132111011	1432111101	1342111110	1324111110
				2314111011
				1234111101
				2134111011
$T_{result} =$ (i=4 and j=1) (i=4 and j=2) (i=4 and j=3)	4312111011	3412111101	3142111110	3124111110
				3214111101
	4132111011	1432111101	1342111110	1324111110
				2314111011
				1234111101
				2134111011

Table 1: Each table shows the DNA strands of the tube T_{result} when executing each iteration in the Algorithm Adjacency_Relation_Comparison. The last table also shows final result after the Algorithm Adjacency_Relation_Comparison is done.

Lemma 4. *After executing Algorithm Adjacency_Relation_Comparison(H,T_{adj}, n_1), $T_{result} = \emptyset$ if $G \ncong H$, and $T_{result} \neq \emptyset$ if $G \cong H$.*

Proof. Straightforward. □

4 A complete algorithm

By combining algorithms described in Section 3, we present Algorithm 3 to solve the subgraph isomorphism problem.

Example 4. After executing the above algorithms, tube $T_{result} = \{4312111011, 3412111101, 3142111110, 3124111110, 3214111101, 4132111011, 1432111101, 1342111110, 1324111110, 2314111011, 1234111101, 2134111011\}$. By applying Read operations in line 4, we obtain twelve DNA strands. This implies that G is isomorphic to H. Moreover, twelve isomorphisms $\pi_1, \pi_2, \ldots, \pi_{12} : V(G) \to V(H)$ are obtained, and they are shown in Table 2.

Theorem 1. *Algorithm Solving_Subgraph_Isomorphism(G, H, n_1, n_2, T_{adj}) solves the subgraph isomorphism problem.*

Algorithm 3 Solving_Subgraph_Isomorphism$(G, H, n_1, n_2, T_{adj})$

Input: (1) G and H are two input graphs, where H is the target graph. (2) n_1 is the number of vertices of G. (3) n_2 is the number of vertices of H. (4) T_{adj} is the tube that contains all n-permutations associated with the adjacency relation of G.

Output: All the isomorphisms from G to H if $G \cong H$. Otherwise, it outputs the message that "G is not isomorphic to H".

1: call Vertex_Extention(H, n_1, n_2) to generate H_e
2: call Adjacency_Relation_Comparision(H_e, T_{adj}, n_1) to obtain T_{result}
3: **if** Detect(T_{result})="yes" **then**
4: Read(T_{result})
5: **else**
6: Output "G is not isomorphic to H"
7: **end if**

Proof. The correctness of this algorithm follows from Lemmas 1, 2, 3, and 4. □

π	$V(G) \to V(H)$
π_1 :	$\pi_1(4) = a,\ \pi_1(3) = b,\ \pi_1(1) = c$
π_2 :	$\pi_2(3) = a,\ \pi_2(4) = b,\ \pi_2(1) = c$
π_3 :	$\pi_3(3) = a,\ \pi_3(1) = b,\ \pi_3(4) = c$
π_4 :	$\pi_4(3) = a,\ \pi_4(1) = b,\ \pi_4(2) = c$
π_5 :	$\pi_5(3) = a,\ \pi_5(2) = b,\ \pi_5(1) = c$
π_6 :	$\pi_6(4) = a,\ \pi_6(1) = b,\ \pi_6(3) = c$
π_7 :	$\pi_7(1) = a,\ \pi_7(4) = b,\ \pi_7(3) = c$
π_8 :	$\pi_8(1) = a,\ \pi_8(3) = b,\ \pi_8(4) = c$
π_9 :	$\pi_9(1) = a,\ \pi_9(3) = b,\ \pi_9(2) = c$
π_{10} :	$\pi_{10}(2) = a, \pi_{10}(3) = b, \pi_{10}(1) = c$
π_{11} :	$\pi_{11}(1) = a, \pi_{11}(2) = b, \pi_{11}(3) = c$
π_{12} :	$\pi_{12}(2) = a, \pi_{12}(1) = b, \pi_{12}(3) = c$

Table 2: Twelve isomorphisms are obtained by aligning the string-representation of T_{result} and the string-representation of H_e. The vertices 1',2', and 3' in H_e are also mapped to the vertices a,b and c in H, respectively.

5 Discussion and Conclusion

In this paper, we present DNA-based algorithms for solving the subgraph isomorphism problem based on biological operations in the Adleman-Lipton model with stickers. Our algorithm can determine not only the two input graphs are isomorphic but also all the isomorphisms between them. The proposed algorithms have two advantages. First, the proposed algorithm actually has a lower rate of errors for hybridization because we develop a computer program to generate good DNA sequences for generating the solution space of the subgraph isomorphism problem. Secondly, the proposed algorithms can be easily performed in a fully automated manner in a laboratory. The full automation manner is essential not only for the speedup of computation but also for error-free computation.

Currently, the future of DNA-based computers is unclear. It is possible that in the future, DNA-based computers will be a good choice for performing massively parallel computations. However, there are still many technical difficulties to overcome before this becomes a reality. We hope that this result can help to demonstrate that DNA-based computing is a technology worth pursuing.

References

[1] L. M. Adleman, "Molecular computation of solutions to combinatorial problems," *Science*, 266:1021–1024, 1994.

[2] Weng-Long Chang and Minyi Guo, "Solving the dominating-set problem in Adleman-Lipton's model," in *Proceedings of the Third International Conference on Parallel and Distributed Computing, Applications and Technologies*, pp. 167–172, 2002.

[3] Weng-Long Chang and Minyi Guo, "Solving the clique problem and the vertex cover problem in Adleman-Lipton's model," in *Proceedings of IASTED International Conference, Networks, Parallel and Distributed Processing, and Applications*, pp. 431–436, 2002.

[4] Weng-Long Chang and Minyi Guo, "Solving NP-complete problem in the Adleman-Lipton model," in *Proceedings of 2002 International Conference on Computer and Information Technology*, pp. 157–162, 2002.

[5] Weng-Long Chang and Minyi Guo, "Resolving the 3-dimensional matching problem and the set-packing problem in Adleman-Liptons model," in *Proceedings of IASTED International Conference, Networks, Parallel and Distributed Processing, and Applications*, pp. 455–460, 2002.

[6] Weng-Long Chang and Minyi Guo, "Solving the set cover problem and the problem of exact 3 cover by 3-sets in the Adleman-Lipton model," *Biosystems*, 72(3):263–275, 2003.

[7] Weng-Long Chang, Minyi Guo, and Michael (Shan-Hui) Ho, "Solving the set-splitting problem in sticker-based model and the Adleman-VLipton model," *Future Generation Computer Systems*, 20(5):875–885, 2004.

[8] Weng-Long Chang, Minyi Guo, and Michael (Shan-Hui) Ho, "Molecular solutions for the subset-sum problem on DNA-based supercomputing," *Biosystems*, 73(2):117–130, 2004.

[9] Weng-Long Chang, Minyi Guo, and Michael (Shan-Hui) Ho, "Fast parallel molecular algorithms for DNA-based computation: factoring integers," *IEEE Transactions on Nanobioscience*, 4(2): 149–163, 2005.

[10] D. Faullhammer, A. R. Cukras, R. J. Lipton, and L. F. Landweber, "Molecular computation: RNA solutions to chess problems," in *Proceedings of National Academica Science U.S.A.* 97, pp. 1385–1389, 2000.

[11] R. P. Feynman, *In Minaturization*, D. H. Gilbert, Ed. New York: Reinhold, 1961, pp. 282–296.

[12] S. D. Gillmor, P. P. Rugheimer, and M. G. Lagally, "Computing with DNA on surfaces," *Surface Science*, 500:699–721, 2002.

[13] F. Guarnieri, M. Fliss, and C. Bancroft, "Making DNA add," Science, 273:220–223, 1996.

[14] Minyi Guo, Michael (Shan-Hui) Ho, and Weng-Long Chang, "Fast parallel molecular solution to the dominating-set problem on massively parallel bio-computing," *Parallel Computing*, 30: 1109–1125, 2004.

[15] R. J. Lipton, "DNA solution of hard computational problems," *Science*, 268:542–545, 1995.

[16] Q. Liu, L. Wang, A. G. Frutos, A. E. Condon, R. M. Corn, and L. M. Smith, "DNA computing on surfaces," *Nature*, 403:175–179, 2000.

[17] Q. Ouyang, P. D. Kaplan, S. Liu, and A. Libchaber, "DNA solution of the maximal clique problem," *Science*, 278:446–449, 1997.

[18] S. Roweis, E. Winfree, R. Burgoyne, N. V. Chelyapov, M. F. Goodman, P. W. K. Rothemund, and L. M. Adleman, "A sticker based model for DNA computation," in *Proceedings of the Second Annual Worksop on DNA Computing*, *DIMACS: Series in Discrete Mathematics and Theoretical Computer Science*, American Mathematical Society, pp. 1–29.

[19] K. Sakamoto, H. Gouzu, K. Komiya, D. Kiga, S. Yokoyama, T. Yokomori, and M. Hagiya, "Molecular computation by DNA hairpin formation," *Science*, 288:1223–1226, 2000.

[20] Sun-Yuan Hsieh and Ming-Yu Chen, "A DNA-based solution to the graph isomorphism problem using Adleman-Lipton model with stickers," *Applied Mathematics and Computation*, 197:672-686, 2008.